

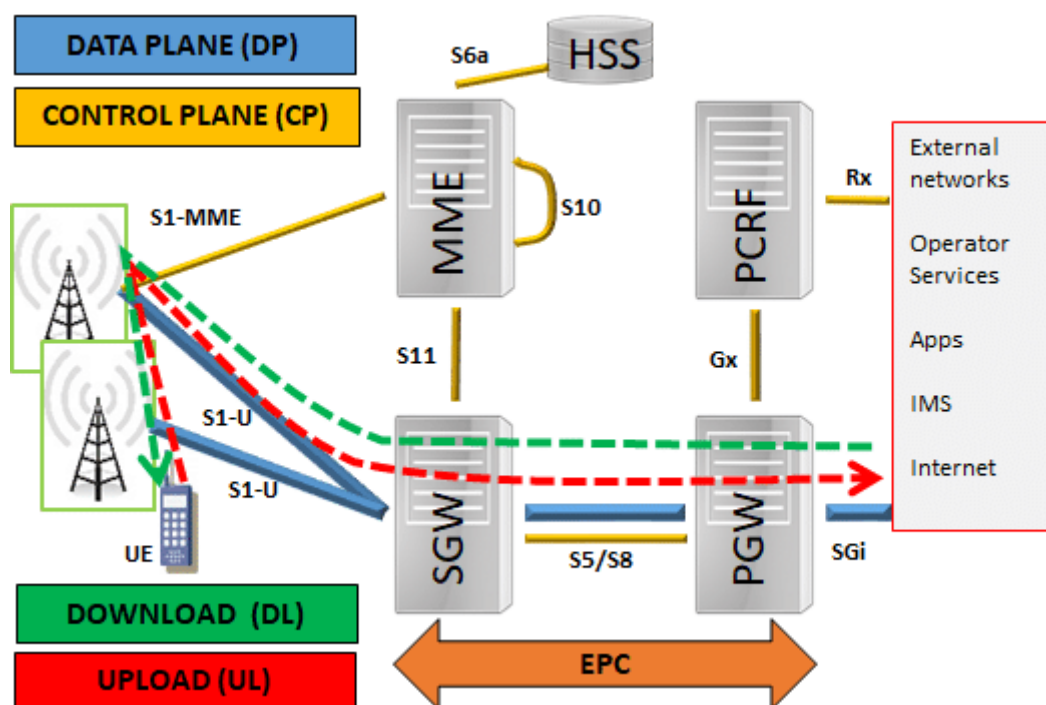
Aula 8 - Interpretar Parâmetros de Core: Principais funções de rede (NFs) no 4G (MME, HSS, SPGW-C, PCRF) e no 5G (NRF, NSSF, AMF, AUSF, UDM, UDR, SMF, PCF)

Objetivo da Aula

Capacitar o aluno a compreender e interpretar os principais parâmetros das funções de rede (Network Functions – NFs) nos Core Networks do 4G e 5G, destacando suas responsabilidades, interações e evolução entre gerações.

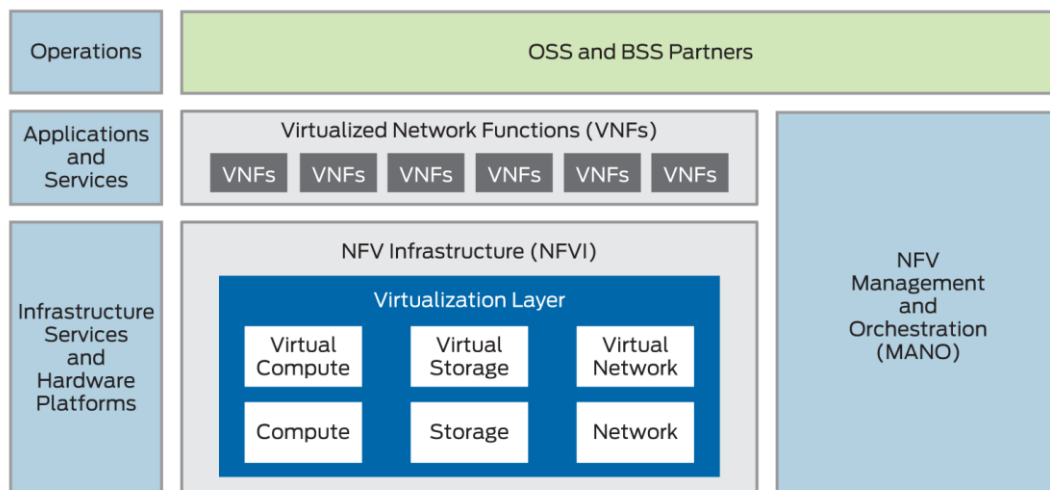
1.1 Arquitetura EPC (4G)

- **EPC (Evolved Packet Core)** é o núcleo das redes 4G LTE. Ele permite comunicação entre UEs (user equipment) e serviços IP, como navegação na internet.

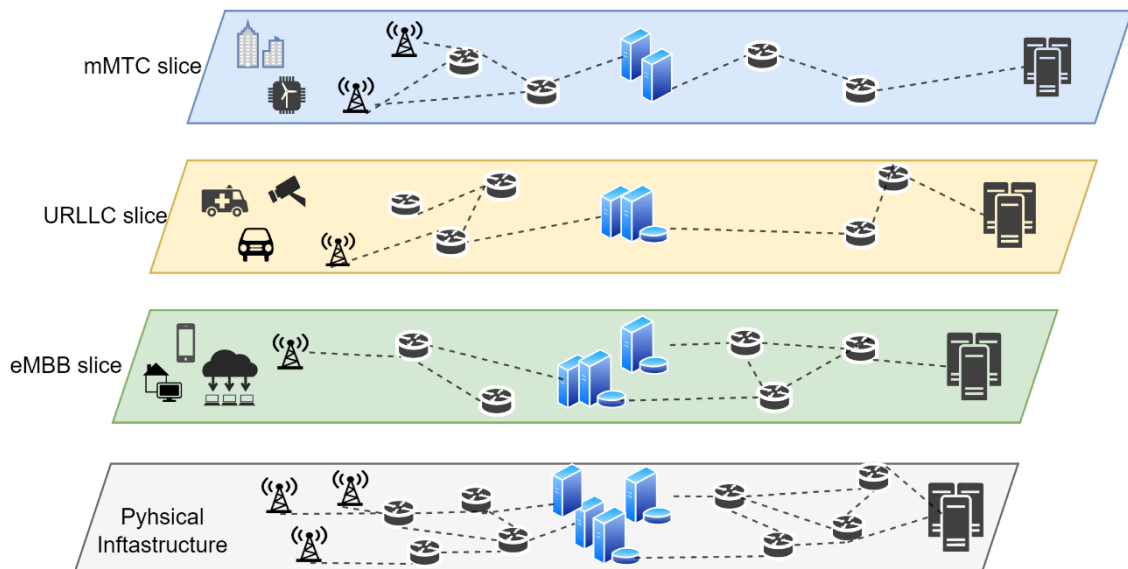


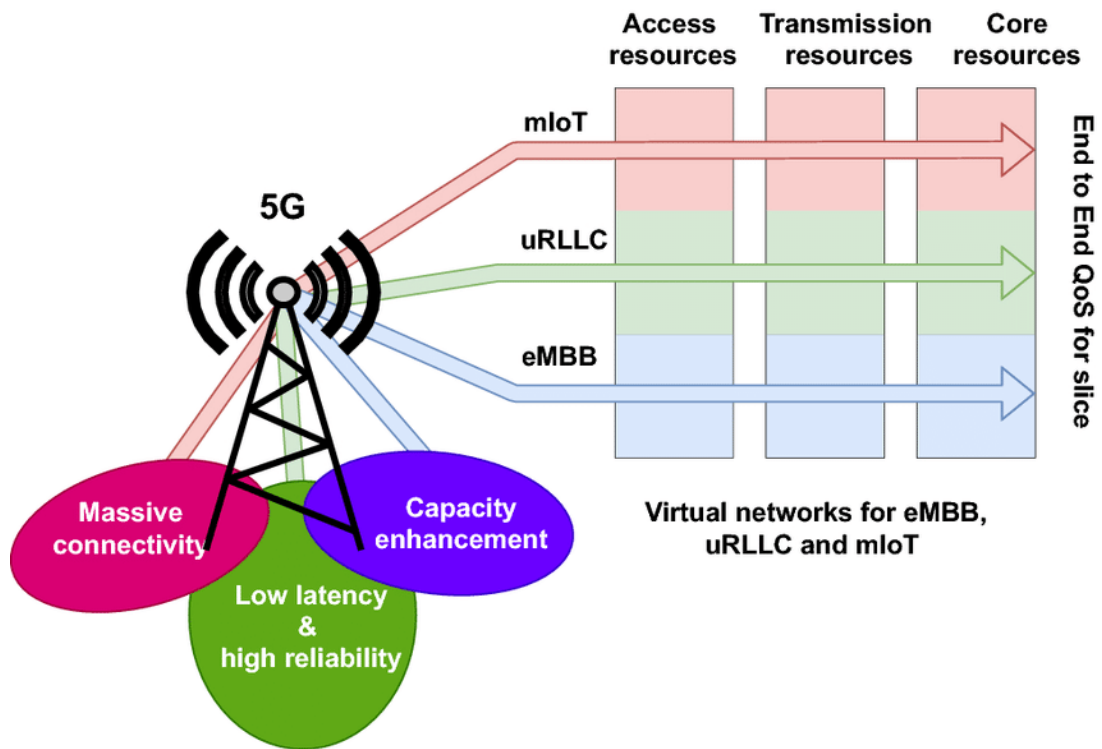
- **Componentes:**
 - MME: controla mobilidade e autenticação.
 - HSS: banco de dados de assinantes.
 - SPGW (dividido em SPGW-C e SPGW-U): roteamento de pacotes.
 - PCRF: aplica políticas e cobrança.
- Utiliza interfaces padronizadas (S1, S5, S11) e tunelamento GTP (GPRS Tunneling Protocol).

1.2 Arquitetura 5GC (5G Core)



- Usa a arquitetura SBA (Service-Based Architecture), com comunicação por APIs RESTful via HTTP/2.
- Cada função de rede é uma Network Function (NF) desacoplada e escalável.
- **Foco em:**
 - Virtualização (NFV),
 - Containers (CNF),
 - Redução de latência,
 - Suporte a serviços diferenciados (IoT, eMBB, URLLC),
 - Network Slicing.

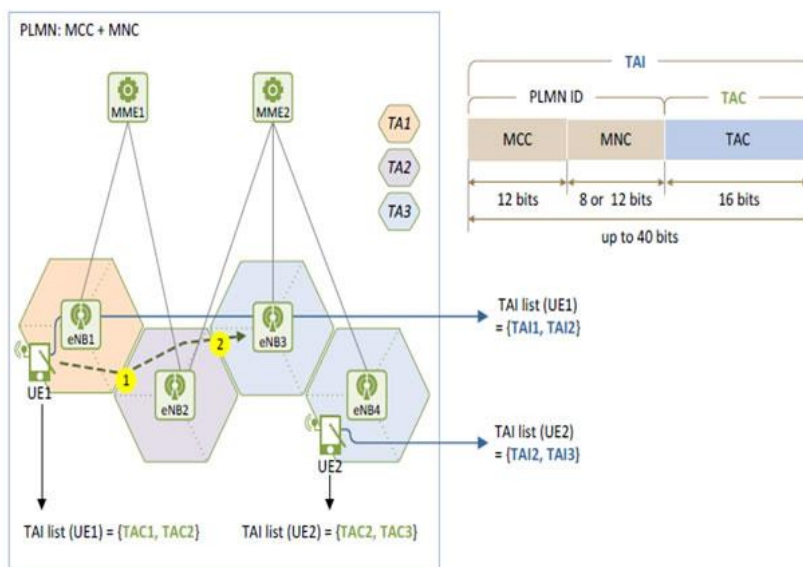




2. Funções de Rede no 4G (EPC)

2.1 MME (Mobility Management Entity)

- Responsável pela mobilidade, autenticação e gerenciamento de sessões NAS.
- Mensagens-chave:
 - Attach Request, Authentication Request, Tracking Area Update.
- **Parâmetros:**
 - IMSI: identifica unicamente o assinante.
 - GUTI: identificador temporário atribuído à UE.
 - TAI (*Tracking Area Identifier*): indica a localização do UE na rede.

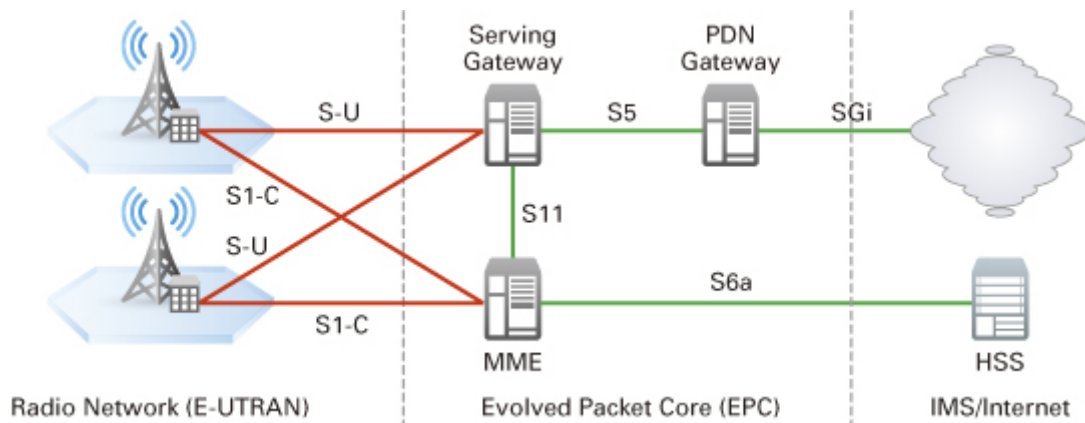


https://www.teleco.com.br/tutoriais/tutorialredeslteident/pagina_8.asp

2.2 HSS (Home Subscriber Server)

- É a base de dados central do assinante.
- Contém:
 - Perfis de serviço (QoS, planos),
 - Chaves criptográficas,
 - Lista de APNs autorizados.

2.3 SPGW-C (Serving Gateway + PDN Gateway – Control Plane)



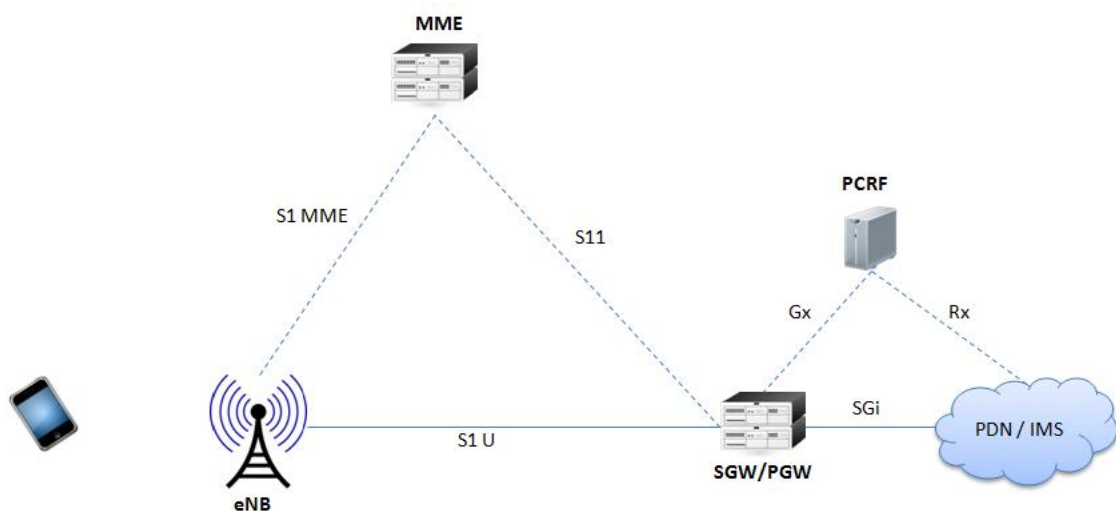
- **Gerencia o caminho do tráfego de dados:**

- Cria túneis GTP,
- Faz NAT e roteamento.

- **Parâmetros:**

- TEID (Tunnel Endpoint ID),
- IP do UE,
- APN utilizado,
- Bearers QoS.

2.4 PCRF (Policy and Charging Rules Function)



- **Define regras de política de uso e cobrança:**

- QoS dinâmico por app ou serviço,
- Tarifação diferenciada.

- **Parâmetros:**

- PCC Rules, SDF (Service Data Flow), Policy Decisions.

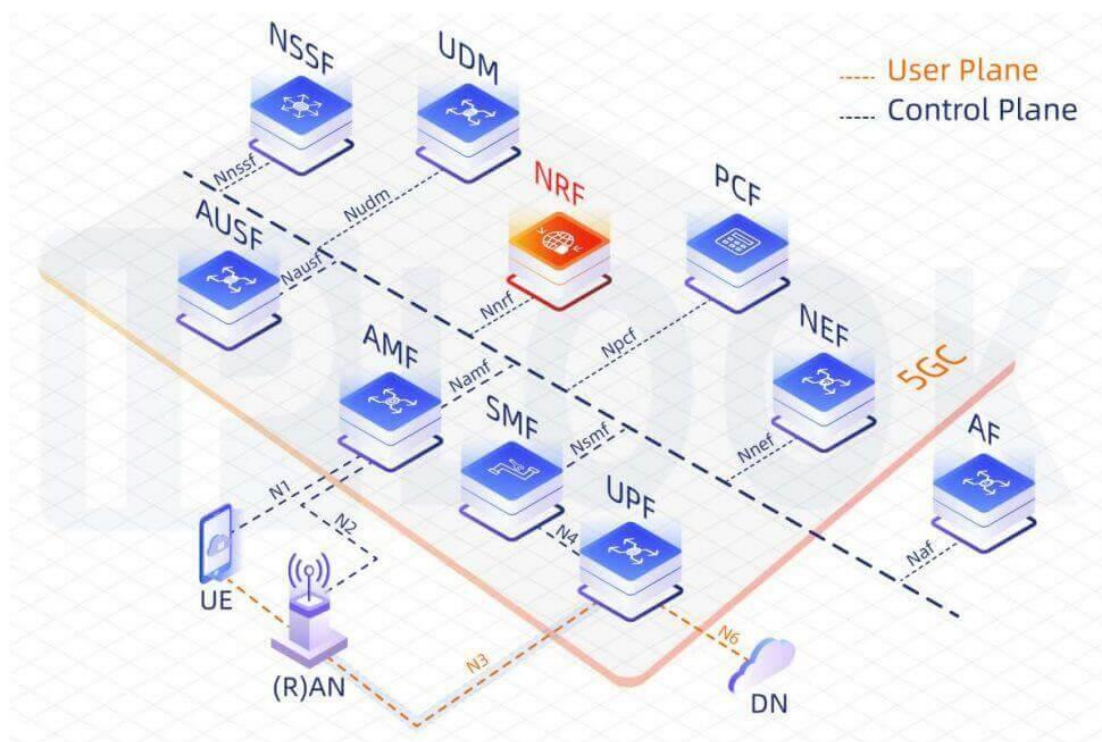
3. Transição para o 5G Core

3.1 Inovações Principais

- Adoção da SBA (Service-Based Architecture):
 - NFs se comunicam por APIs padronizadas.
 - Desacoplamento total de funções de rede.
 - Uso de microserviços, containers, cloud-native.
 - Suporte ao Network Slicing, para diferentes tipos de serviço:
 - IoT (mMTC), missão crítica (URLLC), banda larga (eMBB).
 - Alta disponibilidade e escalabilidade nativa.
-

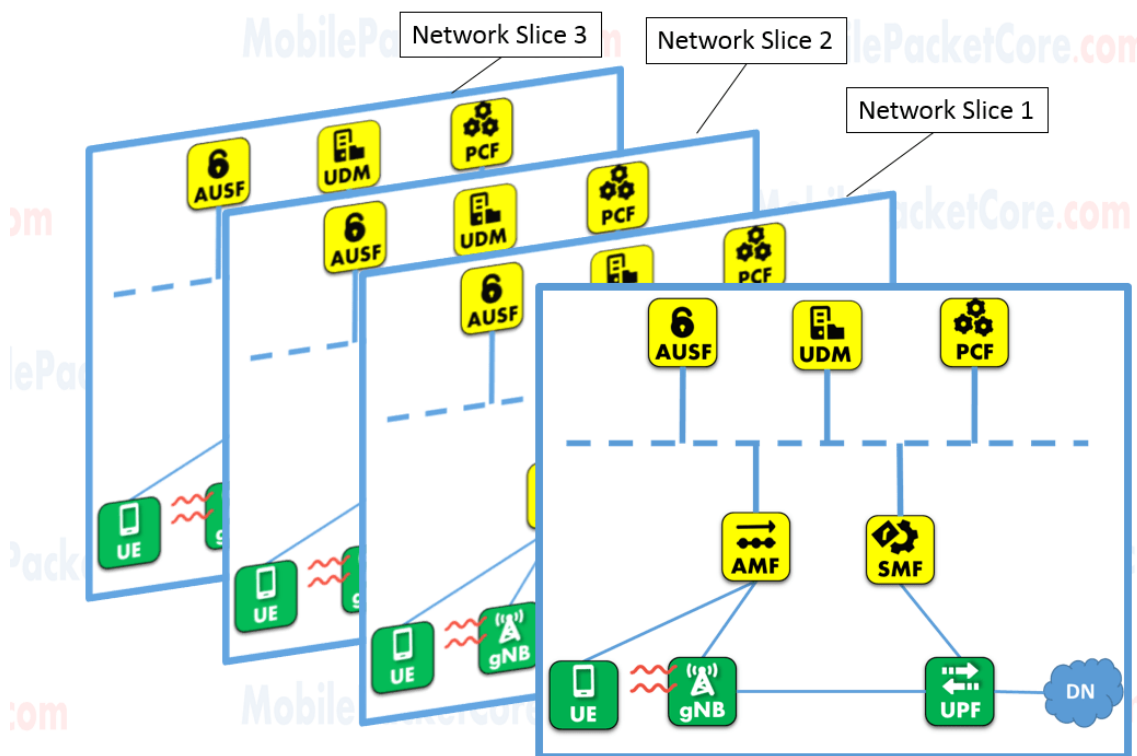
4. Funções de Rede no 5G Core (5GC)

4.1 NRF (Network Repository Function)



- Gerencia o registro e descoberta de NFs na rede.
- Equivalente ao “DNS” dos serviços SBA.
- Informa ao AMF ou SMF quais NFs estão disponíveis.

4.2 NSSF (Network Slice Selection Function)



- Decide qual slice (fatia de rede) o assinante usará.
- Trabalha com o identificador S-NSSAI.

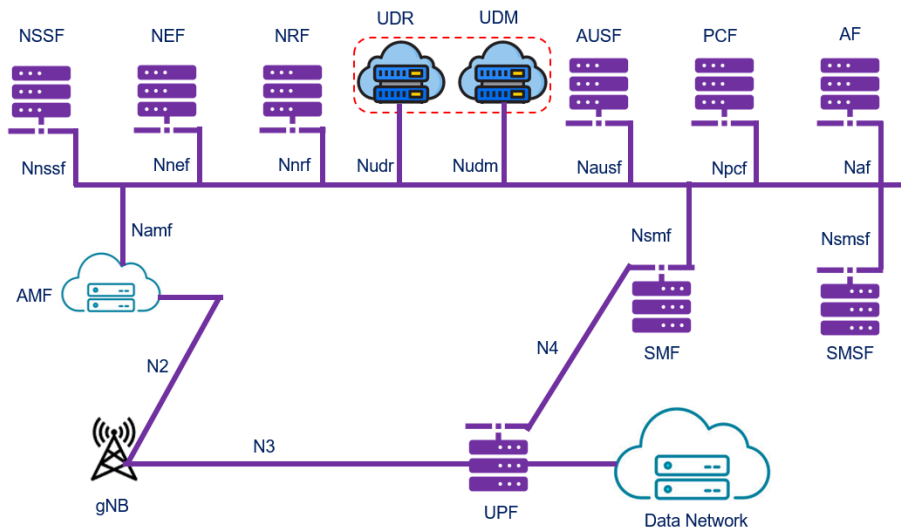
4.3 AMF (Access and Mobility Management Function)

- Substitui o MME no 5G.
- Lida com:
 - Acesso e mobilidade,
 - Conexão inicial, autenticação primária,
 - Parâmetros: SUPI (substitui IMSI), PEI (equipamento), NSSAI.

4.4 AUSF (Authentication Server Function)

- Realiza autenticação do usuário.
- Trabalha em conjunto com o UDM, com base em SUPI e chaves.

4.5 UDM (Unified Data Management)



- Centraliza dados do assinante.
- Responsável por:
 - Chaves,
 - Perfis de política,
 - Dados de sessão.

4.6 UDR (Unified Data Repository)

- Banco de dados genérico compartilhado entre várias NFs.
- APIs REST permitem leitura e escrita de dados.

4.7 SMF (Session Management Function)

- Gerencia sessões PDU (dados).
- Seleciona o UPF (User Plane Function).
- Parâmetros:
 - ID de Sessão PDU,
 - QoS Flow,
 - IP da sessão.

4.8 PCF (Policy Control Function)

- Substitui o PCRF.
- Aplica regras de política unificadas:
 - Controle de acesso,
 - QoS,
 - Regras de cobrança.

◆ 5. Comparativo 4G x 5G

Função	4G (EPC)	5G Core (5GC)
Autenticação	MME + HSS	AMF + AUSF + UDM
Sessões	SPGW-C	SMF + UPF
Políticas	PCRF	PCF
Repositório	HSS	UDR + UDM
Descoberta	—	NRF (descoberta de NFs)
Slicing	—	NSSF (seleção de fatias de rede)
Arquitetura	Baseada em interfaces	SBA com APIs REST (HTTP/2)

ATIVIDADES PRÁTICAS

7.1. Emulação de Rede SDN com Mininet

Objetivo: Simular uma rede com switches OpenFlow e controlador remoto para testar a separação entre plano de controle e de dados.

Requisitos:

- Mininet instalado em Linux Ubuntu (pode usar VM ou WSL)
- Wireshark para captura de pacotes

Passo a Passo:

1. No terminal do Ubuntu, executar:

```
sudo mn --topo=tree,depth=2 --controller=remote --mac --switch=ovsk
```

2. Testar conectividade entre os hosts:

```
pingall
```

3. Capturar pacotes:

```
sudo wireshark &
```

- **Selecionar interfaces s1-ethX, h1-ethX para análise**

4. Verificar comunicação controlada via OpenFlow

Resultado Esperado: Conectividade entre os hosts e observação do controle via SDN.

Atividade Detalhada: Emulação de Topologia LTE com Mininet

1. Preparação do Ambiente

Passo 1: Instalar Mininet no Ubuntu

```
sudo apt update
```

```
sudo apt install mininet
```

Opcional (recomendo para manipulação avançada):

```
sudo apt install openvswitch-switch wireshark tcpdump
```

2. Criar Script Customizado para Topologia LTE

Diretório e arquivo

```
mkdir ~/mininet-lte
```

```
cd ~/mininet-lte
```

```
nano lte_topology.py
```

Exemplo de script lte_topology.py básico para simular MME, HSS e eNodeB

```
from mininet.topo import Topo
from mininet.net import Mininet
from mininet.node import OVSSwitch, RemoteController, Host
from mininet.link import TCLink
from mininet.cli import CLI
from mininet.log import setLogLevel
```

```
class LteTopo(Topo):
```

```
    def build(self):
```

```
        # Criar switch core LTE
```

```
        coreSwitch = self.addSwitch('s1')
```

```
        # Criar hosts representando MME, HSS e eNodeB
```

```
        mme = self.addHost('mme', ip='10.0.0.1/24')
```

```
        hss = self.addHost('hss', ip='10.0.0.2/24')
```

```
        enb = self.addHost('enb', ip='10.0.0.3/24')
```

```

    # Conectar hosts ao switch core

    self.addLink(mme, coreSwitch, bw=10)

    self.addLink(hss, coreSwitch, bw=10)

    self.addLink(enb, coreSwitch, bw=10)

def run():

    topo = LteTopo()

    net = Mininet(topo=topo, link=TCLink, controller=None)

    net.start()

    print("Hosts e IPs configurados:")

    for host in net.hosts:

        print(f"{host.name}: {host.IP()}")

    # Configuração simples de roteamento (se necessário)

    # Exemplo: todos hosts no mesmo segmento, roteamento simples não é obrigatório

    CLI(net)

    net.stop()

if __name__ == '__main__':

    setLogLevel('info')

    run()

```

3. Executar o Script e Testar a Topologia

Rodar o script

```
sudo python3 lte_topology.py
```

No prompt do Mininet, verificar conectividade

Mostrar IPs dos hosts

```
nodes
```

Ping entre hosts

```
pingall
```

Ping específico

```
mme ping hss
```

```
enb ping mme
```

4. Configuração de Regras Básicas de Roteamento e Qualidade de Serviço (QoS)

Exemplo: Limitar banda entre eNodeB e MME usando tc

No prompt do Mininet:

Limitar banda na interface do enb conectada ao switch

```
enb tc qdisc add dev enb-eth0 root tbf rate 5mbit burst 10kb latency 70ms
```

Verificar configuração

```
enb tc qdisc show dev enb-eth0
```

5. Simular Comunicação Entre eNodeB e MME

Simulação simples com netcat para troca de mensagens

No terminal do Mininet CLI:

No host mme, abrir listener TCP na porta 5000

```
mme nc -l 5000
```

Em outro terminal (no Mininet CLI):

No host enb, enviar mensagem para mme na porta 5000

```
enb echo "Hello from eNodeB" | nc 10.0.0.1 5000
```

Você deve ver "Hello from eNodeB" aparecer no terminal do mme.

6. Capturar Tráfego com tcpdump para Análise

Iniciar captura em interface do switch ou hosts

No Mininet CLI:

```
mme tcpdump -i mme-eth0 -w mme_capture.pcap &
```

```
enb tcpdump -i enb-eth0 -w enb_capture.pcap &
```

Parar captura

```
kill %1 # para parar tcpdump no mme
```

```
kill %2 # para parar tcpdump no enb
```

Copiar os arquivos pcap para fora do Mininet para analisar no Wireshark

Em outro terminal Linux (fora do Mininet):

```
sudo cp ~/mininet-lte/mme_capture.pcap ~/Desktop/
```

```
sudo cp ~/mininet-lte/enb_capture.pcap ~/Desktop/
```

Abra os arquivos com Wireshark para analisar tráfego LTE simulado.

Resumo dos comandos principais para a atividade

Passo	Comando/Arquivo
Instalar Mininet	<code>sudo apt install mininet openvswitch-switch</code>
Criar script topologia	<code>nano ~/mininet-lte/lte_topology.py</code>
Rodar topologia	<code>sudo python3 lte_topology.py</code>
Testar ping entre hosts	<code>pingall e.g. mme ping enb</code>
Limitar banda (QoS)	<code>tc qdisc add dev enb-eth0 root tbf rate 5mbit</code>
Simular comunicação netcat	<code>mme nc -l 5000 / ` enb echo "msg"</code>
Capturar tráfego	<code>tcpdump -i ... -w arquivo.pcap</code>

7.2 Laboratório Detalhado: Configuração de VLANs e QoS com Switches Virtuais no Packet Tracer

1. Topologia da Rede

- **Dispositivos:**
 - 3 PCs: PC1, PC2, PC3
 - 1 Switch gerenciável (ex: Cisco 2960)
 - 1 Roteador (ex: Cisco 1941)
- **Conexões:**
 - PC1 conectado à porta FastEthernet 0/1 do switch
 - PC2 conectado à porta FastEthernet 0/2 do switch
 - PC3 conectado à porta FastEthernet 0/3 do switch
 - Switch conectado ao roteador via FastEthernet 0/24 (trunk)

2. Endereçamento IP

Dispositivo	Interface	IP	Máscara	VLAN
PC1	NIC	192.168.10.2	255.255.255.0	10
PC2	NIC	192.168.20.2	255.255.255.0	20
PC3	NIC	192.168.20.3	255.255.255.0	20
Roteador	Fa0/0.10 (subif)	192.168.10.1	255.255.255.0	10
Roteador	Fa0/0.20 (subif)	192.168.20.1	255.255.255.0	20

3. Configuração do Switch Cisco 2960

Switch> enable

Switch# configure terminal

! Criar VLANs

vlan 10

name VOICE

exit

```
vlan 20
```

```
name DATA
```

```
exit
```

! Atribuir portas de acesso

```
interface FastEthernet0/1
```

```
switchport mode access
```

```
switchport access vlan 10
```

```
spanning-tree portfast
```

```
exit
```

```
interface FastEthernet0/2
```

```
switchport mode access
```

```
switchport access vlan 20
```

```
spanning-tree portfast
```

```
exit
```

```
interface FastEthernet0/3
```

```
switchport mode access
```

```
switchport access vlan 20
```

```
spanning-tree portfast
```

```
exit
```

! Configurar trunk para o roteador

```
interface FastEthernet0/24
```

```
switchport mode trunk
```

```
switchport trunk allowed vlan 10,20
```

```
exit
```

4. Configuração do Roteador Cisco 1911/1941

```
Router> enable
```



```
Router# configure terminal
```

```
interface FastEthernet0/0
```

```
no shutdown
```

```
exit
```

! Subinterfaces

```
interface FastEthernet0/0.10
```

```
encapsulation dot1Q 10
```

```
ip address 192.168.10.1 255.255.255.0
```

```
exit
```

```
interface FastEthernet0/0.20
```

```
encapsulation dot1Q 20
```

```
ip address 192.168.20.1 255.255.255.0
```

```
exit
```

5. Configuração dos PCs

PC1

- IP: 192.168.10.2
- Máscara: 255.255.255.0
- Gateway: 192.168.10.1

PC2

- IP: 192.168.20.2
- Máscara: 255.255.255.0
- Gateway: 192.168.20.1

PC3

- IP: 192.168.20.3
 - Máscara: 255.255.255.0
 - Gateway: 192.168.20.1
-

6. ❌ QoS - Corrigido conforme limitações do Switch 2960 no Packet Tracer

💡 O *Packet Tracer* não suporta classes e policieis completas de QoS no 2960.

Você pode simular a confiança no CoS (802.1p) com:

```
Switch(config)# mls qos
```

```
interface FastEthernet0/1
```

```
mls qos trust cos
```

```
exit
```

```
interface FastEthernet0/2
```

```
mls qos trust cos
```

```
exit
```

```
interface FastEthernet0/3
```

```
mls qos trust cos
```

```
exit
```

💡 Alternativa realista e mais visual no PT:

- Criar priorização visual simulando filas com o comando acima.
- Usar ping e análise de tráfego via Simulation Mode.

7. Verificações

! VLANs configuradas

```
Switch# show vlan brief
```

! Porta trunk ativa

```
Switch# show interfaces trunk
```

! IPs configurados no roteador

```
Router# show ip interface brief
```

! Pings

```
PC1> ping 192.168.20.2
```

```
PC2> ping 192.168.10.2
```

! (Opcional) Ping contínuo no Windows

```
PC1> ping -t 192.168.20.2
```

3. Configuração do Switch (maior que >> Cisco 2960)

Entrar no modo privilegiado e configuração global

```
Switch> enable
```

```
Switch# configure terminal
```

Criar VLANs

```
Switch(config)# vlan 10
```

```
Switch(config-vlan)# name VOICE
```

```
Switch(config-vlan)# exit
```

```
Switch(config)# vlan 20
```

```
Switch(config-vlan)# name DATA
```

```
Switch(config-vlan)# exit
```

Configurar portas para VLANs específicas

```
Switch(config)# interface FastEthernet0/1
```

```
Switch(config-if)# switchport mode access
```

```
Switch(config-if)# switchport access vlan 10
```

```
Switch(config-if)# exit
```

```
Switch(config)# interface FastEthernet0/2
```

```
Switch(config-if)# switchport mode access
```

```
Switch(config-if)# switchport access vlan 20
```

```
Switch(config-if)# exit
```

```
Switch(config)# interface FastEthernet0/3
```

```
Switch(config-if)# switchport mode access
```

```
Switch(config-if)# switchport access vlan 20
```

```
Switch(config-if)# exit
```

Configurar porta trunk para roteador

```
Switch(config)# interface FastEthernet0/24  
Switch(config-if)# switchport mode trunk  
Switch(config-if)# exit
```

4. Configuração do Roteador (Cisco 1941)

Entrar no modo privilegiado e configuração global

```
Router> enable  
Router# configure terminal
```

Configurar interface trunk com subinterfaces para cada VLAN

```
Router(config)# interface FastEthernet0/0  
Router(config-if)# no shutdown  
Router(config-if)# exit  
  
Router(config)# interface FastEthernet0/0.10  
Router(config-subif)# encapsulation dot1Q 10  
Router(config-subif)# ip address 192.168.10.1 255.255.255.0  
Router(config-subif)# exit  
  
Router(config)# interface FastEthernet0/0.20  
Router(config-subif)# encapsulation dot1Q 20  
Router(config-subif)# ip address 192.168.20.1 255.255.255.0  
Router(config-subif)# exit
```

5. Configuração dos PCs

- **PC1:**
 - IP: 192.168.10.2
 - Máscara: 255.255.255.0
 - Gateway: 192.168.10.1
- **PC2:**
 - IP: 192.168.20.2
 - Máscara: 255.255.255.0
 - Gateway: 192.168.20.1

- **PC3:**
 - IP: 192.168.20.3
 - Máscara: 255.255.255.0
 - Gateway: 192.168.20.1

6. Configuração de QoS no Switch

Para simplificar no Packet Tracer, usaremos **priorização de filas por VLAN** com comandos básicos.

Entrar na interface do switch

```
Switch# configure terminal
```

Ativar QoS globalmente

```
Switch(config)# mls qos
```

Configurar prioridade alta para VLAN 10 (tráfego de voz)

```
Switch(config)# class-map match-any VOICE
```

```
Switch(config-cmap)# match vlan 10
```

```
Switch(config-cmap)# exit
```

```
Switch(config)# policy-map PRIORITY-VOICE
```

```
Switch(config-pmap)# class VOICE
```

```
Switch(config-pmap)# priority
```

```
Switch(config-pmap)# exit
```

```
Switch(config)# interface range FastEthernet0/1-3
```

```
Switch(config-if-range)# service-policy input PRIORITY-VOICE
```

```
Switch(config-if-range)# exit
```

Obs: O Packet Tracer tem limitações para simular QoS complexa, mas esta configuração básica prioriza tráfego da VLAN 10.

7. Testes e Comandos para Verificação

a) Verificar VLANs no switch

```
Switch# show vlan brief
```

Saída esperada:

VLAN	Name	Status	Ports
10	VOICE	active	Fa0/1
20	DATA	active	Fa0/2, Fa0/3

b) Verificar interfaces trunk

```
Switch# show interfaces trunk
```

Saída esperada:

Port	Mode	Encapsulation	Status	Native vlan
Fa0/24	on	802.1q	trunking	1
Port	Vlans allowed on trunk			
Fa0/24	1-4094			
Port	Vlans allowed and active in management domain			
Fa0/24	10,20			
Port	Vlans in spanning tree forwarding state and not pruned			
Fa0/24	10,20			

c) Verificar configuração IP no roteador

```
Router# show ip interface brief
```

Saída esperada:

Interface	IP-Address	OK?	Method	Status	Protocol
Fa0/0	unassigned	YES	manual	up	up
Fa0/0.10	192.168.10.1	YES	manual	up	up
Fa0/0.20	192.168.20.1	YES	manual	up	up

d) Testar conectividade entre PCs

- No PC1, abrir prompt de comando e executar:

```
ping 192.168.20.2
```

- No PC2, executar:

```
ping 192.168.10.2
```

e) Ping contínuo para medir latência (Windows)

No PC1 (Windows):

```
ping -t 192.168.20.2
```

Em Linux:

```
ping 192.168.20.2
```

8. Resultado Esperado

- PCs nas VLANs diferentes conseguem pingar entre si via roteador (inter-VLAN routing).
- Tráfego da VLAN 10 (voz) deve apresentar prioridade na fila QoS do switch, resultando em menor latência e jitter — perceptível em análise de tráfego real.
- A configuração de QoS minimiza latência do tráfego de voz, mesmo com tráfego simultâneo na VLAN 20.

7.3 - Simulação de Rede LTE com Packet Tracer

Objetivo: Simular a comunicação entre um UE (User Equipment), uma estação rádio-base (eNodeB) e um servidor EPC (Core LTE).

Topologia:

[UE (Smartphone)] <---> [eNodeB (Roteador)] <---> [EPC (Servidor)]		
10.0.0.2	10.0.0.1 192.168.0.1	192.168.0.100

Passo a Passo Detalhado:

1. Abrir o Cisco Packet Tracer

- Inicie o software Cisco Packet Tracer.

2. Inserir os dispositivos na área de trabalho

- Clique em **End Devices** → arraste 1 **Smartphone**.
- Clique em **Network Devices** → **Routers** → arraste 1 roteador **Generic (1841 ou 2911)**.
- Clique em **End Devices** → arraste 1 **Server**.

3. Conectar os dispositivos com cabos

- Clique em **Connections** (ícone do relâmpago):
 - Selecione **cabo cobre direto** (cabo laranja).
 - Conecte:
 - **Smartphone** → porta **Wi-Fi0** ao **Router** → porta **FastEthernet0/0**
 - **Router** → porta **FastEthernet0/1** ao **Servidor** → porta **FastEthernet0**

4. Configurar os IPs

No Smartphone (UE):

1. Clique no **Smartphone** → **Config** → **Wireless0**:
 - Configure:
 - IP Address: 10.0.0.2
 - Subnet Mask: 255.255.255.0
 - Default Gateway: 10.0.0.1

No Roteador (eNodeB):

1. Clique no **Router** → **Config** → **FastEthernet0/0**:

- IP Address: 10.0.0.1
- Subnet Mask: 255.255.255.0
- Status: Ligado

2. Vá para **FastEthernet0/1**:

- IP Address: 192.168.0.1
- Subnet Mask: 255.255.255.0
- Status: Ligado

No Servidor (EPC):

1. Clique no **Server** → **Config** → **FastEthernet0**:

- IP Address: 192.168.0.100
- Subnet Mask: 255.255.255.0
- Default Gateway: 192.168.0.1

5. Criar rotas estáticas no roteador

1. Clique no **Router** → **CLI** → pressione ENTER.

2. Digite os comandos:

```
enable
configure terminal
ip route 10.0.0.0 255.255.255.0 FastEthernet0/0
ip route 192.168.0.0 255.255.255.0 FastEthernet0/1
end
write memory
```

6. Testar conectividade

Do Smartphone (UE):

1. Clique no **Smartphone** → **Desktop** → **Command Prompt**:

- Digite:

```
ping 192.168.0.100
```

- Resultado esperado: quatro respostas com tempo (latência) em ms.

2. Se disponível, execute:

```
tracert 192.168.0.100
```

Do Servidor (EPC):

1. Clique no **Server** → **Desktop** → **Command Prompt**:
 - Teste a volta:

```
ping 10.0.0.2
```

Resultado Esperado:

- O Smartphone (UE) consegue **pingar o EPC**, passando pelo roteador (eNodeB).
- O **tracert** mostra os saltos da comunicação.
- Toda a rede LTE simulada se comporta como um backbone básico da arquitetura LTE.